

LennyPM

Instant, actionable product management advice from 320+ Lenny's Podcast episodes.

Product Requirements Document
RAG Application — Cohort 49 Assignment

April 2026

1. Product Overview & Problem Statement

Product Name & One-Liner

LennyPM — Instant, actionable product management advice from 320+ Lenny's Podcast episodes.

User Persona

Raj, 27, is a PM-1 and the sole product manager at an early-stage AI startup (~30 employees). He manages a small cross-functional team of 3-4 developers and 2 UX designers. Raj spent two years as a software developer straight out of college before completing his MBA to transition into product management.

Despite his technical background, Raj faces a steep learning curve. As the only PM, he owns the entire product roadmap with no senior PM to shadow or consult. The AI landscape moves fast — his CEO expects rapid feature shipping to stay competitive, and every prioritization call feels high-stakes.

When Raj hits an unfamiliar challenge — pushing back on scope creep, structuring his first product strategy, or deciding what to cut from a release — **he has no playbook.** He Googles the problem, asks ChatGPT, or reaches out to former classmates, but gets generic advice that doesn't account for the messy realities of a resource-constrained startup. He's heard that experienced PMs like Shreyas Doshi, Lenny Rachitsky, and others have discussed exactly these situations on Lenny's Podcast — but with 320+ episodes averaging 60-90 minutes each, finding the right insight feels like searching for a needle in a haystack made of other needles.

Raj doesn't need more content. He needs the right answer, from a trusted source, right when he's stuck.

Current Workflow & Pain Points

Step	What Raj Does	Pain Point
1	Encounters an unfamiliar PM challenge at work	—
2	Googles the topic → gets blog posts, Medium articles	Generic, contradictory, hard to judge credibility
3	Asks ChatGPT → gets a reasonable-sounding answer	No real-world context, may hallucinate frameworks
4	Asks former MBA classmates or seniors	Slow, depends on availability
5	Remembers Lenny's Podcast → opens Spotify/YouTube	—
6	Searches keywords → gets episode titles, not answers	Search only matches titles, not what was said inside
7	Picks a promising episode → skips around timeline	No way to know where the relevant advice lives
8	Listens for 15-20 min → may or may not find it	Huge time investment with uncertain payoff
9	Tries another episode → repeat steps 6-8	Compounding frustration and time loss
10	Gives up or decides with low confidence	Ships with doubt, risks a bad call

The pain points cluster around **steps 6-9**. The content Raj needs exists inside Lenny's Podcast — the problem is there is no way to search inside episodes, connect advice across episodes, or get a direct answer without manually listening through hours of audio.

Why RAG? — Decision Tree Reasoning

Why not Prompt Engineering alone?

Lenny's Podcast spans 320+ episodes — roughly 3-5 million words of transcript data. This far exceeds any LLM's context window. Even with large-context models, stuffing this volume of messy, conversational text would lead to degraded attention, increased hallucination, and prohibitively expensive API calls. Prompt engineering works when your knowledge fits in a single prompt. This doesn't.

Why not Fine-Tuning?

Raj doesn't need the model to *behave* differently — he doesn't need a custom tone or domain-specific reasoning style. He needs the model to *find and surface specific advice* from a trusted knowledge base. Fine-tuning changes how a model thinks; RAG changes what a model knows. This is a retrieval problem, not a behavior problem.

Why RAG is the right fit

The content already exists and is highly credible — Lenny's Podcast is one of the most trusted sources in the PM community, featuring practitioners like Shreyas Doshi, Marty Cagan, and Julie Zhuo. The problem isn't content quality — it's content *accessibility*. RAG lets us index, chunk, and retrieve precise segments from 320+ episodes so Raj gets the exact advice he needs, cited back to the source episode, in seconds instead of hours. As new episodes release, the knowledge base simply grows — no retraining needed.

2. User Stories & Use Cases

User Stories

1. "As an early-career PM, I want to ask how experienced PMs run roadmap reviews and get cited advice from podcast episodes, so that I can align my team on deliverables and timelines before my first review meeting."
2. "As an early-career PM, I want to ask how to handle stakeholder disagreements and get specific techniques discussed by PM practitioners, so that I can push back on my CEO's requests with confidence and data."
3. "As an early-career PM, I want to ask how to translate a product vision into a strategy document, so that I can write my first product strategy with a proven framework instead of guessing."
4. "As an early-career PM, I want to compare what multiple podcast guests have said about the same topic (e.g., prioritization), so that I can see different perspectives and pick the approach that fits my context."
5. "As an early-career PM, I want every answer to cite the specific episode and guest it came from, so that I can listen to the full conversation for deeper context when needed."
6. "As an early-career PM, I want to ask follow-up questions within the same conversation, so that I can drill deeper into a topic without re-explaining my situation."
7. "As an early-career PM, I want the system to clearly tell me when a topic wasn't covered in the podcast, so that I don't receive made-up advice that sounds credible but has no source."

Primary Use Cases

Use Case 1: Direct Question → Actionable Answer

- **Happy path:** Raj asks "How should I run my first sprint planning?" → LennyPM retrieves relevant segments from 2-3 episodes, synthesizes a step-by-step answer, and cites each source.
- **Edge case:** Raj asks a vague query like "help me with planning" → System asks a clarifying question.

Use Case 2: Cross-Episode Comparison

- **Happy path:** Raj asks "What do different guests say about saying no to stakeholders?" → LennyPM retrieves advice from multiple guests and presents them side-by-side with citations.
- **Edge case:** Only one episode covers the topic → System returns that single source and states it clearly.

Use Case 3: "I Don't Know" Scenario

- **Happy path:** Raj asks "What does Lenny say about managing a sales team?" → Topic was never covered → LennyPM states this clearly and suggests alternatives.
- **Edge case:** Topic is partially related → System surfaces the partial match with a disclaimer.

Negative Use Cases

- **Should NOT summarize entire episodes.** Full episode summaries risk copyright issues and don't align with the product's purpose of task-specific retrieval.
- **Should NOT fabricate advice when no source exists.** The product's core value is trusted, sourced advice. Hallucinated answers destroy that trust instantly.

Prioritization

Priority	Stories	Justification
P0 — Must Have	#1, #2, #3, #5, #7	Core value prop. Without these, the product has no reason to exist.
P1 — Should Have	#4, #6	Differentiators that make LennyPM better than 'ChatGPT with transcripts.'
P2 — Nice to Have	Bookmarking, sharing, topic browsing	Engagement features for retention — not needed for v1.

3. Technical Architecture

Architecture Diagram — Full RAG Pipeline

Data Sources (320 .txt transcripts) → Pre-processing (clean filler, add metadata) → Chunking (paragraph + 10-20% overlap) → Embedding (OpenAI text-embedding-3-large) → Vector DB (Pinecone) → User Query → Retrieval (similarity search, top-K=10) → Re-ranking (Cohere Rerank, top-K→5) → Augmented Prompt (query + chunks + system prompt) → LLM Generation (GPT-4o-mini) → Response with Citations

Observability layer (LangSmith) traces every stage of the pipeline for debugging and improvement.

Component	Choice	Justification
Data Sources	320 .txt transcript files	Plain text, no OCR needed. Pre-process to strip filler, normalize speakers, attach metadata.
Chunking	Paragraph-level, 300-500 tokens, 10-20% overlap	Preserves natural thought boundaries. Overlap ensures concepts bridging paragraphs aren't lost.
Embedding	OpenAI text-embedding-3-large (3,072 dim)	Higher-fidelity semantic capture for nuanced PM concepts. One-time indexing cost (~\$0.81).
Vector DB	Pinecone (managed)	No ops overhead. Metadata filtering (guest, episode, topic). Sub-100ms queries.
Retrieval	Cosine similarity, top-K=10	Wide initial net. Re-ranker filters to top 5 for precision.
Re-ranking	Cohere Rerank 3.5	Cross-encoder accuracy. Reads query + chunk together for deeper relevance scoring.
LLM	GPT-4o-mini (temp 0.2)	Right-sized for synthesis. Retrieval does the heavy lifting; LLM synthesizes and cites.
Observability	LangSmith	Native LangChain integration. Full pipeline tracing per query.
Frontend	Streamlit (v1)	Fast to build, sufficient for v1 chat interface.

Context Window Budget

Component	Estimated Tokens
System prompt	~300
Top 5 chunks (avg 400 tokens each)	~2,000
User query	~50
Total input	~2,350
Output (answer + citations)	~500-800
Total per request	~2,850-3,150

GPT-4o-mini supports 128K tokens — our usage is well within limits, leaving room for multi-turn conversations.

4. RAG Failure Points & Mitigation

This section covers all 9 failure points in the LennyPM RAG pipeline — each specific to podcast transcripts, with detection via observability, fix, and the PM trade-off it forces.

Failure 1: Data Quality / OCR

What breaks:

Podcast transcripts from speech-to-text are messy: filler words ("um," "you know") inflate chunks without semantic value; speaker names are misattributed; PM-specific terms ("RICE framework," "OKRs," "Marty Cagan") are garbled; punctuation is inconsistent.

How we detect it:

LangSmith trace inspection of retrieved chunks for garbled text. Embedding quality audit — chunks with unusually low similarity scores may indicate noise. User feedback correlation — thumbs-down mapped to source chunks.

How we fix it:

Pre-processing pipeline strips filler using a conversational stopword list. Normalize speaker labels against a known guest roster. Domain-specific spell-check on PM terminology. Quality threshold — chunks below coherence score are flagged.

PM decision it forces:

Coverage vs. quality. Aggressive cleaning risks removing legitimate conversational nuance. For v1, we lean toward aggressive cleaning with manual review — a smaller, higher-quality index beats a noisy comprehensive one when trust is the core value.

Failure 2: Chunking Strategy

What breaks:

Podcasts have no headers or sections. A guest's 4-step framework gets split between chunks. Examples are orphaned from the concepts they illustrate. Context-dependent pronouns ("That's exactly what we did at Stripe") lose their referent.

How we detect it:

LangSmith traces showing partial answers. Low re-ranking scores after high cosine similarity (chunks match keywords but lack coherent answers). Follow-up query patterns ("can you give me the full framework?").

How we fix it:

Increase overlap to 20% where traces show split-context issues. A/B test 300-token vs. 500-token chunks on retrieval accuracy. Store `previous_chunk_id` and `next_chunk_id` in Pinecone metadata for adjacent chunk retrieval.

PM decision it forces:

Precision vs. context. Smaller chunks are more precise in retrieval but lose context. The PM must find the optimal chunk size through evaluation data and accept that no single size is perfect for every query type.

Failure 3: Embedding Quality

What breaks:

Synonym mismatch — user asks about "feature prioritization" but guest used "backlog grooming." PM jargon gaps — domain-specific meanings not captured. Register gap — formal queries vs. conversational transcripts. Negation sensitivity — "when NOT to launch" placed near "when to launch."

How we detect it:

Retrieval precision testing with test Q&A pairs. LangSmith similarity score distributions — consistently low scores (< 0.7) suggest model underperformance. Manual spot checks of top-5 chunks for 10 common PM queries.

How we fix it:

Query expansion — LLM expands query with synonyms before embedding. Hybrid search — combine vector similarity with BM25 keyword search. Fine-tuned embeddings (v2) if persistent domain gaps remain.

PM decision it forces:

General-purpose vs. domain-specific embeddings. Fine-tuned embeddings improve accuracy but add engineering cost. The PM must decide if query expansion and hybrid search close the gap sufficiently for v1.

Failure 4: Search / Retrieval

What breaks:

Popular topics ("product-market fit") flood top-K with shallow mentions. Wrong top-K — too few misses best chunks, too many passes noise downstream. Metadata filter failures from inconsistent guest names. No recency weighting.

How we detect it:

Recall@10 measurement against gold-standard chunks. LangSmith trace analysis — are retrieved chunks substantive or shallow mentions? Re-ranker drop rate — if 8/10 chunks are dropped, retrieval is too noisy.

How we fix it:

Top-K tuning starting at K=10, adjusted by re-ranker retention rate. Metadata normalization during ingestion. Optional recency boost (time-decay weight on similarity scores).

PM decision it forces:

Breadth vs. precision. Higher top-K gives the re-ranker more material but increases latency and cost. The PM must balance recall against response time targets (< 3 seconds).

Failure 5: Re-ranking Failures

What breaks:

Cross-episode confusion — chunks from different episodes match on surface text but differ in relevance. Length bias — longer chunks score higher regardless of quality. Adversarial topic mixing — chunks that mention the query topic in passing ("unlike prioritization, which we discussed earlier...") get moderate scores.

How we detect it:

LangSmith re-rank trace comparison (before vs. after order). Generation quality vs. re-rank score correlation. A/B testing with and without re-ranker to measure marginal value.

How we fix it:

Chunk size normalization to 300-500 tokens. Minimum relevance score threshold — send only chunks that pass, not always top 5. Multi-pass re-ranking for complex queries (v2).

PM decision it forces:

Cost vs. quality. The re-ranker adds ~100-200ms latency and ~\$0.002 per query. If A/B testing shows minimal improvement, the PM must decide whether to keep it. For LennyPM, where trust is core, we keep it.

Failure 6: Prompt / Augmentation

What breaks:

Context dilution — 5 chunks from 5 episodes create noisy context. Instruction drift — LLM ignores guardrails. "Lost in the middle" effect — LLM pays more attention to first and last chunks. Conflicting guest advice requires careful handling.

How we detect it:

LangSmith prompt inspection on thumbs-down responses. Hallucination rate tracking — claims not in any source chunk. Citation accuracy audit (20 responses/week).

How we fix it:

Prompt engineering iteration on test Q&A pairs. Chunk ordering by re-rank score (best first and last). Explicit conflict handling instruction. Dynamic chunk count — fewer chunks when scores drop sharply.

PM decision it forces:

Synthesis vs. raw retrieval. Should the LLM merge advice or present perspectives separately? Synthesis is more actionable but risks misrepresenting nuance. For v1, we synthesize with per-claim citations and add "compare perspectives" mode in v2.

Failure 7: Model Quality

What breaks:

GPT-4o-mini may produce surface-level comparisons for complex multi-step queries. Overkill for simple metadata lookups. Tone may feel too formal vs. the conversational source material. Citation formatting may be inconsistent.

How we detect it:

LLM-as-Judge (GPT-4o evaluating GPT-4o-mini, weekly sample of 50). Query complexity classification — track quality by simple/moderate/complex tier. User satisfaction correlation by query type.

How we fix it:

Query routing (v2): metadata search for lookups, GPT-4o-mini for moderate, GPT-4o for complex queries. Tone tuning in system prompt. Structured output (JSON mode) for citation consistency.

PM decision it forces:

Cost vs. capability. GPT-4o is ~10x more expensive. If 80% of queries are moderate and 20% are complex, routing only complex queries to GPT-4o keeps average cost low while maintaining quality where it matters.

Failure 8: Data Drift

What breaks:

New weekly episodes not ingested — users asking about recent topics get outdated answers. PM advice from 2021 may reference outdated tools/frameworks. Guest career changes make attributions stale. Embedding model updates break compatibility.

How we detect it:

Ingestion pipeline alerts if no new episodes for 14+ days. Freshness dashboard — track chunk timestamp distribution in results. Embedding version tracking in metadata. User queries about recent topics returning no results.

How we fix it:

Automated weekly ingestion via cron job. Staleness metadata — disclaimer on answers from episodes older than 18 months. Re-embedding schedule when model updates (~\$0.81 cost). New episodes auto-processed but flagged for review.

PM decision it forces:

Automation vs. manual curation. Full automation risks ingesting low-quality transcripts. Manual curation creates bottlenecks. For v1, we automate with a quality check step — PM approves new batches weekly in < 15 minutes.

Failure 9: User Behavior (Non-Technical)

What breaks:

Vague queries ("help," "strategy") — single words match everything and nothing. Non-PM queries from users who see a chat interface. Distrust of AI answers. Conversational expectations ("tell me more about what she said after that"). Copy-pasted long messages embed poorly.

How we detect it:

Query length distribution in LangSmith. Out-of-scope query rate (> 20% signals scope confusion). Session depth analysis. Citation click-through rate.

How we fix it:

Onboarding with 3-4 example queries. Clarification prompt for queries < 4 words. Graceful out-of-scope handling. Input preprocessing — extract core question from long pastes. Trust-building via collapsible "view source" section.

PM decision it forces:

Guided vs. open-ended experience. Heavy onboarding feels restrictive; too little and users flounder. For v1, we lean guided. As usage data reveals natural query patterns, we relax constraints in v2.

5. Data Permissions & Access Control

LennyPM is a **single-user, consumer-facing product** — not an enterprise tool. The knowledge base consists entirely of publicly available podcast content, not proprietary documents. There is no scenario where different users need different access levels. This is fundamentally different from enterprise RAG systems like Glean, where permissions must be inherited from source platforms and enforced via metadata filtering at query time.

LennyPM does not need permission-based access control because there is no sensitive, role-specific, or confidential data in the knowledge base.

Data Privacy Considerations

User Query Privacy

Queries are logged in LangSmith for observability but anonymized — no user identity attached. 90-day retention, then auto-purged. Users can opt out via settings toggle.

Content Licensing

The system synthesizes advice and cites sources — it never reproduces full transcript passages. Functionally similar to a search engine surfacing snippets. If LennyPM scales, a licensing agreement with Lenny Rachitsky would be required.

Data Storage Security

Vector embeddings stored in Pinecone (SOC 2 Type II compliant, encrypted at rest and in transit). Raw transcripts stored locally during processing, not exposed through the application. No PII collected in v1.

6. Evaluation & Success Metrics

Retrieval & Generation Quality

Category	Metric	Target
Retrieval	Precision@5 — relevant chunks in top 5	≥ 70%
Retrieval	Recall@10 — gold chunk in top 10	≥ 85%
Retrieval	MRR (Mean Reciprocal Rank)	≥ 0.6
Generation	Accuracy — correctly represents guest advice	≥ 90%
Generation	Faithfulness — every claim grounded in source	≥ 95%
Generation	Citation correctness	≥ 95%
Policy	Hallucination rate	< 5%
Policy	Out-of-scope compliance	≥ 95%
Business	Time saved per query	15-20 min
Business	Sessions per user per week	≥ 3
Business	Query-to-satisfaction rate (thumbs-up)	≥ 70%

Evaluation Method

Primary — LLM-as-Judge: GPT-4o evaluates GPT-4o-mini's responses weekly (50 samples) on accuracy, faithfulness, completeness, and citation correctness (1-5 scale). Like having a senior PM review a junior PM's work.

Secondary — Human audit: Monthly manual review of 30 randomly sampled responses. Catches subtle quality issues that LLM-as-Judge may miss.

Test Q&A Pairs (15 pairs)

#	Test Query	Expected Behavior
1	How do I run my first sprint planning?	Return step-by-step advice with citations
2	What does Shreyas Doshi say about prioritization?	Filter by guest, return Shreyas-specific advice
3	How do I say no to my CEO?	Techniques with real examples from guests
4	What frameworks exist for product strategy?	Compare frameworks from different guests
5	How do I write a good PRD?	Structured advice with citations
6	What's the weather today?	Decline — out of scope
7	Summarize episode 150	Decline — not designed for summaries
8	How do I handle a product launch that flopped?	Relevant advice, handle emotional nuance
9	Compare Marty Cagan and Lenny on roadmaps	Both perspectives with separate citations
10	How do I deal with technical debt?	PM-framed advice (not engineering advice)
11	What should I do about my divorce?	Decline — out of scope
12	How do I transition from engineering to PM?	Relevant career transition advice
13	Tell me about hiring PMs	Specific criteria and frameworks discussed
14	What is product-market fit?	Synthesize across guests, note differing definitions
15	How do I set OKRs for a small team?	Actionable advice scoped to small teams

Escalation Design

Low-confidence: When re-ranker's top score < 0.3, append disclaimer and suggest PM communities. **No results:** State clearly and suggest alternatives. **Feedback loop:** Thumbs-down responses queued for review; patterns trigger pipeline investigation.

7. Token Economics & Cost

All pricing sourced from official API documentation as of April 2026.

Cost Per Query — Breakdown

Component	Model / Service	Unit Price	Cost Per Query
Query embedding	text-embedding-3-large	\$0.13 / 1M tokens	\$0.0000065
Vector search	Pinecone Starter	\$70/mo (amortized)	~\$0.003
Re-ranking	Cohere Rerank 3.5	\$2.00 / 1K searches	\$0.002
LLM input (~2,350 tokens)	GPT-4o-mini	\$0.15 / 1M tokens	\$0.000353
LLM output (~600 tokens)	GPT-4o-mini	\$0.60 / 1M tokens	\$0.000360
Total per query			\$0.006

Cost Per User Per Month

Assuming a power user queries 5 times/day, 5 days/week (~100 queries/month):

Metric	Value
API cost per user per month	~\$0.60
Pinecone (amortized)	~\$0.70
LangSmith (free tier < 5K traces)	\$0
Total cost per user per month	~\$1.30

Model Selection Trade-Off

Scenario	Model	Cost/Query	When to use
Default (80%)	GPT-4o-mini	\$0.006	Single-topic questions, framework lookups
Complex (20%)	GPT-4o	\$0.035	Cross-episode comparison, multi-step reasoning
Simple lookups (future)	Metadata only	~\$0.003	"What episode was X on?" — no LLM needed

Blended cost per query: $0.8 \times \$0.006 + 0.2 \times \$0.035 = \sim\$0.012$

Cost Optimization Strategies

- **Prompt caching:** GPT-4o-mini cached input at \$0.075/1M tokens — 50% discount on repeated system prompts.
- **Query routing:** Simple lookups → metadata search (no LLM). Moderate → GPT-4o-mini. Complex → GPT-4o.
- **Response caching:** Cache frequent queries. 50 identical questions pay for 1 LLM call.
- **Batch re-indexing:** OpenAI Batch API at 50% discount for new episode embedding.

8. UX, Rollout & GTM

Interface Type: Web-Based Chat

Streamlit (v1), custom React (v2). Raj's workflow is browser-based — a web chat meets him where he already is. Chat is the most natural pattern for Q&A. Centered layout, max-width 700px, dark/light theme, mobile-responsive. Example queries shown on first load for onboarding.

Source Citations & Confidence Handling

Inline citations: Every claim tagged with [Ep. #, Guest Name]. **Source panel:** Collapsible section below each response showing transcript excerpts used. **Deep link (v2):** Link to source episode on Spotify/YouTube.

Confidence Level	Behavior
High (re-rank > 0.7)	Deliver answer normally with citations
Medium (0.3 - 0.7)	Deliver with disclaimer: "This answer may not fully address your question."
Low (< 0.3)	"This topic doesn't appear to be well-covered in the podcast."
Out of scope	"I'm designed to help with PM questions based on Lenny's Podcast."

Launch Phases

Phase	Audience	Success Criteria
Alpha (Weeks 1-2)	5-10 PM peers from MBA network	≥ 60% satisfaction, < 10% hallucination rate
Beta (Weeks 3-6)	50-100 PMs via Lenny's Newsletter Slack	≥ 70% satisfaction, < 5% hallucination, < 5s response time
GA (Month 3+)	Open access, Product Hunt launch	≥ 75% satisfaction, ≥ 3 sessions/user/week, < 3% hallucination

Launch Risks

Risk	Impact	Mitigation
Hallucination goes viral	PM shares wrong answer on Twitter	Low temp (0.2), strict grounding, monitoring
Content licensing dispute	Lenny's team objects	Synthesize only (never verbatim). Proactive outreach during Alpha
Poor first impression	Users try once, never return	Onboarding examples, validate top-20 queries before launch

9. Future Scope — Fine-Tuning & Agentic RAG

When Would Fine-Tuning Help?

Fine-tuning becomes valuable when the product needs to change *how* the model communicates, not just *what* it retrieves. If user feedback shows responses feel too "textbook," fine-tuning on PM writing could add a practitioner voice. If LennyPM becomes an official Lenny product, fine-tuning could align with Lenny's personal tone.

Approach

PEFT/LoRA over full fine-tuning: Adapts model behavior with a fraction of the parameters. Like adding a PM-specific lens to the camera rather than buying a new camera. **Base model:** Fine-tune an open-source SLM (Llama 3 8B or Mistral 7B) to reduce inference cost and API dependency. **Distillation:** Use GPT-4o to generate high-quality responses as training data for the smaller model — an apprentice learning from a master.

Agentic RAG Roadmap

- **Web search integration:** When the podcast doesn't cover a topic, search the web with appropriate caveats.
- **Multi-source RAG:** Expand beyond the podcast to Lenny's Newsletter, guest blog posts, and approved PM resources.
- **Proactive suggestions:** Surface new episodes matching the user's recent query patterns.
- **Action-taking (v3+):** Generate draft roadmap templates or stakeholder communication plans using retrieved frameworks.

10. AI Specs — The Build Brief

Platform Target

Custom web app (Python + LangChain backend, Streamlit frontend for v1). Lovable and Google AI Studio lack support for custom RAG pipelines with Pinecone, Cohere Rerank, and LangSmith. A custom build gives full control over every component.

Complete System Prompt (380 words)

You are LennyPM, an AI product management advisor powered by insights from 320+ episodes of Lenny's Podcast – one of the most trusted sources in the PM community.

ROLE: You help early-career product managers get actionable, specific advice for their day-to-day PM challenges. You are like a senior PM colleague who has listened to every episode and can instantly recall the most relevant insights.

CONSTRAINTS:

- Answer **ONLY** using information from the provided context (retrieved podcast chunks).
- **NEVER** generate advice from your general knowledge. If the context does not contain relevant information, say: "This topic doesn't appear to be covered in the podcast episodes I have access to."
- **NEVER** fabricate episode numbers, guest names, or quotes.
- **NEVER** summarize entire episodes.

CITATION FORMAT:

- Every claim must cite its source: [Ep. {number}, {Guest Name}]
- When multiple guests discuss the same topic, present each perspective separately with its own citation.
- When only one source is relevant, cite it clearly and do not fabricate additional perspectives.

TONE: Warm, practical, and conversational – like a helpful PM colleague, not a textbook. Use concrete examples from the podcast when available.

OUTPUT FORMAT:

- Lead with the most actionable advice first. Use short paragraphs, not bullet-point walls.
- For frameworks: Framework Name → Steps → When to use → When NOT to use
- For comparisons: Present perspectives side by side with clear attributions.

HANDLING UNCERTAINTY:

- Partially relevant context: Share what you have with a disclaimer.
- Vague queries (fewer than 4 words): Ask a clarifying question before retrieving.

GUARDRAILS:

- Do not answer questions outside of product management.
- Do not express personal opinions – only relay what podcast guests have said.
- Do not engage with offensive, harmful, or inappropriate queries.

Knowledge Base Specification

Attribute	Detail
Source	Lenny's Podcast (lennypodcast.com)
Format	320 plain text files (.txt)
Estimated size	~3-5 million words (~6.25M tokens)
Topics	Strategy, prioritization, hiring, stakeholder mgmt, growth, PMF, metrics, user research
Update frequency	Weekly (new transcripts within 48 hours of publication)
Notable guests	Shreyas Doshi, Marty Cagan, Julie Zhuo, Gibson Biddle, Gokul Rajaram, 300+ others

Functional Requirements

MUST Have (P0)

- Accept natural language queries about PM topics
- Retrieve relevant chunks via RAG pipeline (embedding → vector search → re-ranking)
- Generate answers grounded exclusively in retrieved chunks
- Cite every claim with episode number and guest name

- Clearly state when a topic is not covered (no hallucination)
- Respond in under 5 seconds; support follow-up questions

SHOULD Have (P1)

- Compare advice from multiple guests on the same topic
- Ask clarifying questions for vague queries
- Thumbs-up/thumbs-down feedback mechanism
- Collapsible "Sources" section; dark/light theme; onboarding examples

COULD Have (P2)

- Bookmark answers; share snippets; filter by guest/topic; deep link to episodes

5 Example Interactions

Interaction 1: Direct Question (Happy Path)

User: How should I run my first roadmap review?

LennyPM: Start by framing the review around outcomes, not features. As Gibson Biddle suggests, structure your roadmap around key metrics [Ep. 72, Gibson Biddle]. Share the document 24 hours in advance [Ep. 134, Lenny Rachitsky]. Focus on the top 3 priorities — save lower-priority items for async feedback [Ep. 89, Shreyas Doshi].

Interaction 2: Cross-Episode Comparison

User: What do different guests say about saying no to stakeholders?

LennyPM: Shreyas Doshi recommends "disagree and commit" [Ep. 45]. Marty Cagan argues PMs should push back with data — show opportunity cost [Ep. 102]. Julie Zhuo suggests having the hard conversation early and privately [Ep. 167]. All three agree: saying no without offering an alternative is a dead end.

Interaction 3: "I Don't Know" Scenario

User: What does Lenny say about managing a sales team?

LennyPM: This topic doesn't appear to be covered in the podcast episodes I have access to. If you're looking for how PMs should *collaborate with sales teams*, I can check if that's been discussed — want me to look into that?

Interaction 4: Vague Query (Clarification)

User: Help

LennyPM: Happy to help! Could you be more specific? Are you looking for advice on product strategy, stakeholder management, team management, or something else?

Interaction 5: Low Confidence / Escalation

User: How should I structure compensation for my PM team?

LennyPM: I found some related content, but I'm not very confident in this answer. PM compensation was briefly mentioned in a career paths discussion [Ep. 203]. For specific compensation data, check Levels.fyi or Pave.

Error Handling

Scenario	System Response
No results	"This topic doesn't appear to be covered. Try rephrasing or asking a related PM topic."
API timeout (> 10s)	"Taking longer than usual. Please try again." + auto-retry once
Out-of-scope query	"I'm designed for PM questions based on Lenny's Podcast. I don't have info on [topic]."
Offensive input	"I'm not able to respond to that. I'm here to help with PM questions."
Service outage	"Experiencing technical difficulties. Please try again shortly." + PM alert via LangSmith